

Desafio 1 · O mesmo esqueleto, outro negócio

Uma aplicação de várias telas, com dados mockados, para o nicho que o grupo sortear

1. O que é este desafio

Nas aulas 3 a 7 construímos o KiOferta junto: estrutura de pastas, roteador por hash, telas como módulos, menu gerado da lista de rotas, CSS por componente, Flexbox, find e filter. Agora cada grupo vai aplicar **o mesmo esqueleto a outro problema**, sorteado em aula.

O negócio muda. A arquitetura, não. Ao final, qualquer pessoa deve conseguir abrir o projeto de qualquer grupo e saber onde está cada coisa, porque todos os projetos têm a mesma forma.

O que se espera, em uma frase

Um aplicativo de seis telas, com dados mockados, que responde a uma pergunta prática do tema sorteado, construído no mesmo padrão do KiOferta e entregue com um relatório sobre a construção.

O que este desafio não é

Não tem backend, banco de dados nem login de verdade. Todos os dados ficam em arquivos de dados mockados dentro do projeto, e nada precisa sobreviver a um F5.

Não é para usar CSS Grid, framework CSS nem framework de JavaScript. O layout inteiro sai de Flexbox.

Não é para copiar o KiOferta trocando as palavras. O tema sorteado tem perguntas próprias, e é isso que o relatório precisa mostrar.

2. O sorteio

Os temas são sorteados em aula, um por grupo, sem repetição e sem troca. Todos têm a mesma dificuldade: alguém publica um registro, outra pessoa procura e compara.

Tema	O problema que ele resolve	O que alguém publica
Carona universitária	Quem faz o mesmo trajeto no mesmo horário não tem como se encontrar.	Uma carona, com saída, horário e vagas.
Achados e perdidos	O objeto perdido vai parar em um balcão que ninguém sabe onde fica.	Um item achado, com local e data.
Adoção de animais	Protetores anunciam em grupos de mensagem, e o anúncio se perde no dia seguinte.	Um animal para adoção, com porte e cidade.
Feira do bairro	O cliente não sabe quais produtores estarão na feira de sábado.	Uma banca, com produtos e dia.
Troca de livros	O livro da disciplina já cursada fica parado na estante de alguém.	Um livro disponível, com estado de conservação.
Vagas de estágio	A vaga circula por e-mail e some antes de chegar a quem procura.	Uma vaga, com requisito, bolsa e bairro.
Aulas particulares	Quem precisa de reforço não sabe quem dá aula daquela matéria.	Uma oferta de aula, com matéria e valor por hora.
Serviços do bairro	Eletricista, diarista e montador dependem de indicação boca a boca.	Um serviço, com área de atendimento.
Quadras e campos	Descobrir qual está livre no sábado exige ligar para cinco lugares.	Um horário disponível, com valor.
Agenda cultural	A programação da cidade está espalhada em páginas diferentes.	Um evento, com data, local e preço.
Marmitas do dia	O cardápio muda todo dia e ninguém sabe o que tem antes de ligar.	Um cardápio do dia, com preço e entrega.
Mutirões e voluntariado	A ação social não encontra voluntário, e o voluntário não encontra a ação.	Um mutirão, com data e o que precisa.

3. O mapa de equivalência

Antes de escrever a primeira linha, traduza o KiOferta para o seu tema. Esta tabela entra no relatório, preenchida com o tema do grupo. O exemplo abaixo usa carona universitária.

No KiOferta	No seu tema	No exemplo (carona)
Produto	O que a pessoa procura	Um trecho: Mogi das Cruzes para a Fatec
Oferta	O registro publicado por alguém	Uma carona de amanhã, às 7h
Mercado	Quem oferece, ou onde acontece	O motorista e o ponto de saída
Preço	O número que decide a escolha	O valor da vaga
Distância	O segundo critério de decisão	O horário de saída
Contribuinte	Quem publica e precisa de conta	O motorista cadastrado
Visitante	Quem só consulta, sem cadastro	O aluno procurando carona

A pergunta que o seu aplicativo responde

O KiOferta não existe para listar preços: existe para responder se vale a pena ir até lá. Escreva em uma frase a pergunta equivalente do seu tema, e deixe essa frase decidir o que aparece primeiro em cada tela. Ela abre o relatório.

4. A estrutura obrigatória

Igual à do projeto que construímos. Mesmos nomes de pasta, mesmo papel de cada arquivo.

```
src/
├── index.html      uma página só, com #app e o menu
├── css/
│   ├── tokens.css  as variáveis de cor, tinta e espaçamento
│   ├── base.css    reset e o que vale para o site inteiro
│   └── style.css    importa os anteriores
├── js/
│   ├── main.js     roteador: hashchange, find na lista de rotas
│   ├── rotas/rotas.js  a lista única de telas
│   ├── navbar/navbar.js  o menu gerado a partir da lista
│   ├── sessao/sessao.js  entrar, sair e usuarioAtual
│   ├── dadosMockados/  os arrays de dados do seu tema
│   └── paginas/       uma pasta ou arquivo por tela, com o CSS dela
```

- **E1.** Cada tela é um módulo que exporta um objeto com url, label, icon e pagina, e entra em rotas.js. Tela que não aparece no menu usa label vazio.
- **E2.** O menu é gerado a partir da lista de rotas. Nenhum item escrito à mão no HTML.
- **E3.** Um arquivo de CSS por componente, importado pelo módulo da tela. Nada de CSS inline nem de um arquivo único gigante.

- **E4.** Dados mockados com, no mínimo, **12 registros**, **4 publicadores** e **4 usuários**. Todo registro tem id, e o registro guarda o id de quem publicou.
- **E5.** Vale usar o repositório do KiOferta como modelo, inclusive copiando a estrutura. O que não vale é entregar o KiOferta com os nomes trocados: os dados, as telas e os textos são do seu tema.

5. As seis telas

Tela	O que ela faz	O que ela exercita
1 · Início	Campo de busca dominante e as categorias do seu tema como segundo caminho.	Flexbox, e o termo digitado chegando na próxima tela.
2 · Resultados	A lista do que foi encontrado, com dois critérios de ordenação e estado vazio.	filter, map com join, e o teste .length === 0.
3 · Detalhe	Um registro escolhido, com quem publicou e os dados que decidem a escolha.	find pelo id vindo da lista.
4 · Publicar	O formulário de quem publica um registro novo, validado pelo próprio HTML.	Formulário, find para recusar repetido e push.
5 · Minha conta	Login simples, os dados de quem entrou e os registros publicados por ele. Com botão sair.	find no login, filter pelo id do usuário, módulo de sessão.
6 · Rota inexistente	Qualquer endereço que não existe cai aqui, com uma saída para o início.	O find do roteador devolvendo undefined.

Regras de layout, para as seis telas

L1. Todo alinhamento sai de Flexbox. Nenhuma tela usa display: grid.

L2. Nenhum filho de contêiner flex tem width ou height fixo. Quando o tamanho importar, use flex: 0 0 <medida>.

L3. As cores vêm de tokens.css. Nenhuma cor escrita direto na regra, e uma única cor de destaque por tela.

L4. Tudo precisa funcionar em **360px de largura**, sem rolagem horizontal.

6. O relatório

De três a cinco páginas, em PDF, com estas seções na ordem. É sobre a construção, não sobre o tema: quem lê quer entender como vocês chegaram ao resultado.

- **R1 · O problema e quem usa.** A pergunta que o aplicativo responde, e três situações de uso, como no estudo de caso do KiOferta.
- **R2 · O mapa de equivalência.** A tabela da seção 3 preenchida com o tema do grupo.
- **R3 · A estrutura.** A árvore de arquivos do projeto e, em uma linha cada, o papel dos principais.
- **R4 · Três decisões e um descartado.** Cada decisão com o motivo em duas ou três linhas. O descartado traz a alternativa que vocês consideraram e por que não ficou.
- **R5 · A divisão de tarefas.** A tabela da seção 7, preenchida, com os números dos commits de cada integrante.
- **R6 · Duas dificuldades reais.** O erro que apareceu, o que vocês acharam que era, e o que era de fato.
- **R7 · O que ficou de fora.** O que o aplicativo ainda não faz, e por quê. Lacuna assumida não tira ponto; lacuna escondida, sim.

Capturas

Inclua as seis telas em 360px, mais a mensagem de erro do login e o estado vazio da busca.

7. A divisão de tarefas

Grupo de três, uma semana. A divisão abaixo é uma sugestão que funciona; o grupo pode mudar, desde que a tabela do relatório mostre a divisão real.

Integrante	Responsabilidade principal	Entregáveis
A	Dados e descoberta	Dados mockados, tela de início e tela de resultados, com busca e ordenação.
B	Detalhe e publicação	Tela de detalhe, formulário de publicar, validações e o aviso de registro repetido.
C	Conta e fundação	tokens.css e base.css, módulo de sessão, tela de conta e tela de rota inexistente.

- Cada integrante escreve o **CSS da própria tela**. É o que evita três pessoas no mesmo arquivo.

- `rotas.js` e `navbar.js` são o ponto de encontro: combinem a assinatura das telas no primeiro dia e não mexam nelas sozinhos depois.
- **Todos precisam ter commits próprios no histórico.** A nota da divisão de tarefas é individual: integrante sem nenhum commit não recebe essa parte, mesmo que o grupo entregue tudo.
- Se alguém do grupo parar de responder, avisem **antes** da entrega, não depois.

8. Entrega

- Repositório **público** no GitHub, criado pelo grupo, com os três integrantes aparecendo no histórico.
- README.md com o tema sorteado, os integrantes e como rodar (`npm install` e `npm run dev`).
- O relatório em PDF, dentro do repositório, em `docs/`.
- O link do repositório postado no ambiente da disciplina até a data de entrega. Entrega atrasada perde 1,0 ponto por dia.

9. Apresentação

Oito minutos por grupo, com o aplicativo rodando. Sem slides.

Etapa	Tempo	O que mostrar
1. O problema	1 min	O tema sorteado e a pergunta que o aplicativo responde, em uma frase.
2. O fluxo	2 min	Buscar, abrir um detalhe, publicar um registro e entrar na conta, tudo em 360px.
3. O código	3 min	Onde estão o <code>find</code> e o <code>filter</code> , explicados por quem escreveu. Cada integrante fala da sua parte.
4. A decisão	1 min	Uma decisão do relatório e o que foi descartado por causa dela.
5. A pergunta	1 min	O professor aponta um elemento e pergunta: esta regra é do contêiner pai ou do filho? Mostre no DevTools.

10. Critérios de avaliação

Critério	O que se espera	Pontos
Estrutura no padrão	Mesma organização do modelo: rotas, módulos de tela, menu gerado, CSS por componente e tokens.	2,0
As seis telas	Todas presentes, navegação funcionando e sobrevivendo ao F5, incluindo a rota inexistente.	2,0
find, filter e map	Busca filtrando, detalhe encontrando pelo id, conta listando os registros do usuário e estado vazio correto.	2,0
Layout em Flexbox	Sem Grid, sem largura fixa em filho, funcionando em 360px e com a cor de destaque usada uma vez por tela.	1,5
Relatório	As sete seções, com o mapa de equivalência preenchido, as decisões, o descartado e as dificuldades reais.	1,5
Divisão e apresentação	Commits de todos no histórico, tabela batendo com o git log, e as cinco etapas dentro de oito minutos.	1,0
Total		10,0

Bônus (+0,5)

Um segundo critério de ordenação que o usuário escolhe, ou um filtro por categoria que funciona junto com a busca por texto, sem recarregar a tela.

11. Antes de entregar, confira

- As seis telas abrem pelo menu ou por link, e o F5 em qualquer endereço leva à tela certa ou à de rota inexistente.
- A busca sem resultado mostra a mensagem de vazio, e não uma lista em branco.
- Todo `.map()` termina em `.join("")`, e nenhum `var(--alguma-coisa)` aponta para um token que não existe.
- Nenhum filho de contêiner flex tem `width` fixo, e nada rola para o lado em 360px.
- O git log mostra commits dos três integrantes, e a tabela do relatório bate com ele.
- O projeto roda em uma máquina limpa com `npm install` e `npm run dev`.